Exercicio 1 : EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';

BD_registro_carros/postgres@PostgreSQL 18

No limit

Query | Query History

```sql
DROP TABLE IF EXISTS carro, pessoa;
CREATE TABLE IF NOT EXISTS pessoa (
id_pessoa INTEGER PRIMARY KEY,
nome VARCHAR(100) NOT NULL,
nascimento DATE
);
CREATE TABLE IF NOT EXISTS carro (
id_carro INTEGER PRIMARY KEY,
placa CHAR(7) NOT NULL,
ano INTEGER,
id_pessoa INTEGER NOT NULL,
FOREIGN KEY (id_pessoa)
REFERENCES pessoa(id_pessoa)
ON DELETE CASCADE
);


EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```

Data Output | Messages | Notifications

| | QUERY PLAN<br>text |
|---|---|
| 1 | Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=3.912..9.437 rows=5.00 loop... |
| 2 | Filter: ((nome)::text = 'Ana Silva'::text) |
| 3 | Rows Removed by Filter: 99995 |
| 4 | Buffers: shared hit=688 |
| 5 | Planning Time: 0.086 ms |
| 6 | Execution Time: 9.456 ms |

CREATE INDEX idx_pessoa_nome ON pessoa (nome);

BD_registro_carros/postgres@PostgreSQL 18

No limit

Query | Query History

```sql
DROP TABLE IF EXISTS carro, pessoa;
CREATE TABLE IF NOT EXISTS pessoa (
id_pessoa INTEGER PRIMARY KEY,
nome VARCHAR(100) NOT NULL,
nascimento DATE
);
CREATE TABLE IF NOT EXISTS carro (
id_carro INTEGER PRIMARY KEY,
placa CHAR(7) NOT NULL,
ano INTEGER,
id_pessoa INTEGER NOT NULL,
FOREIGN KEY (id_pessoa)
REFERENCES pessoa(id_pessoa)
ON DELETE CASCADE
);


EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';

CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```

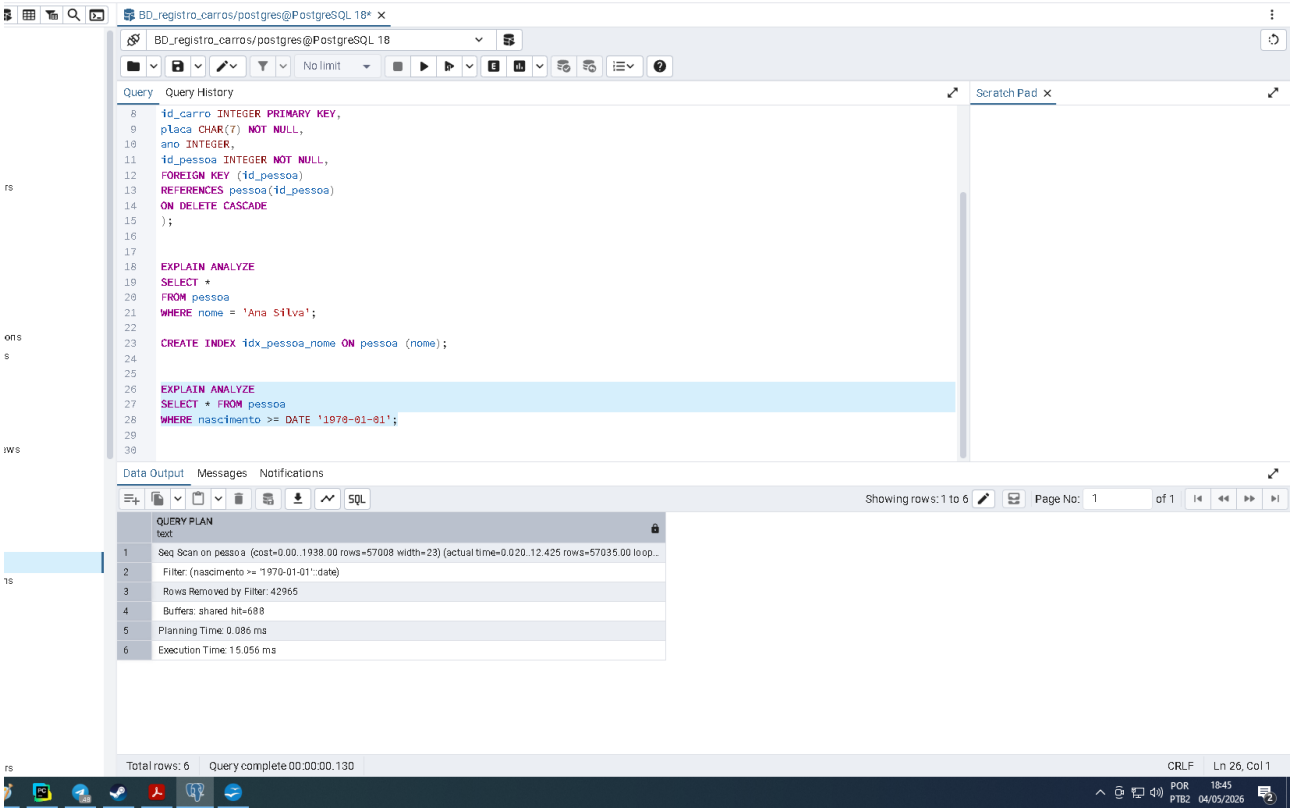
Data Output | Messages | Notifications

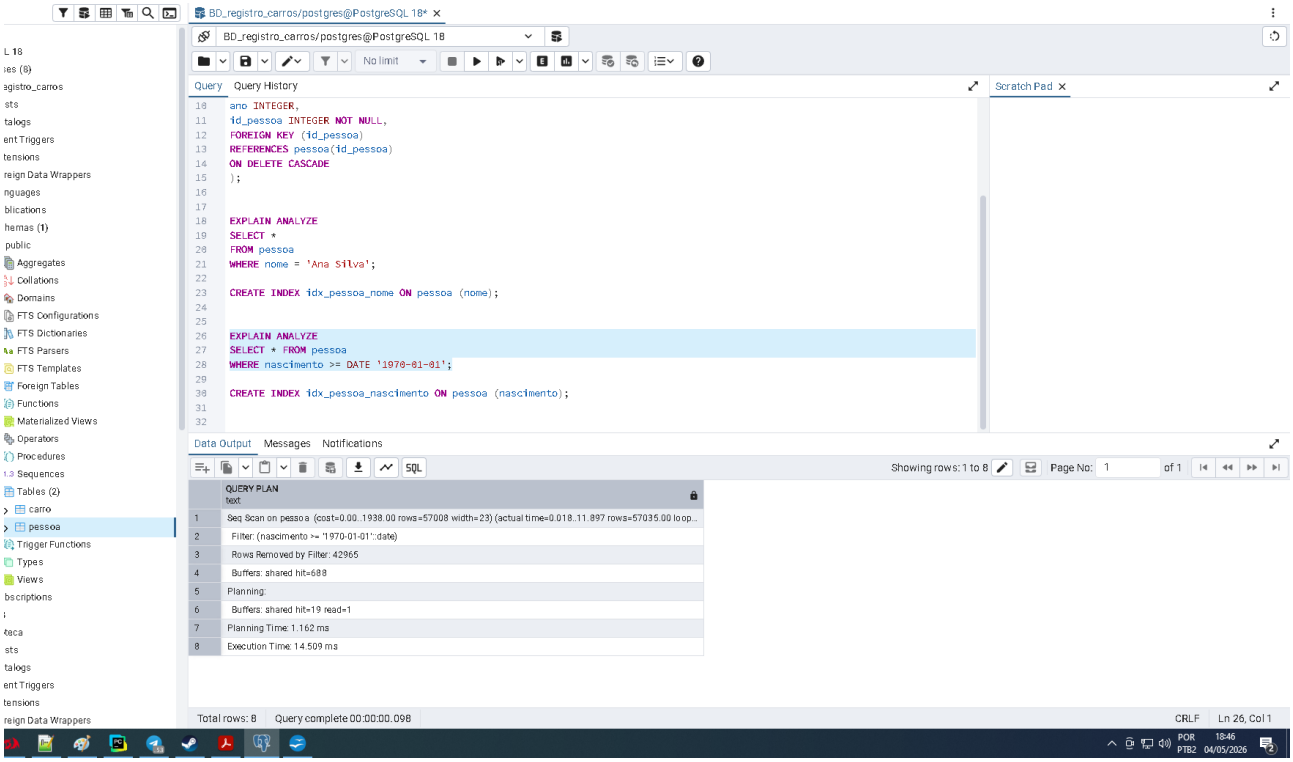
| | QUERY PLAN<br>text |
|---|---|
| 1 | Bitmap Heap Scan on pessoa (cost=4.34..26.73 rows=6 width=23) (actual time=0.495..0.500 rows=5.00 loops=1) |
| 2 | Recheck Cond: ((nome)::text = 'Ana Silva'::text) |
| 3 | Heap Blocks: exact=5 |
| 4 | Buffers: shared hit=5 read=2 |
| 5 | -> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.473..0.473 rows=5.00 loo... |
| 6 | Index Cond: ((nome)::text = 'Ana Silva'::text) |
| 7 | Index Searches: 1 |
| 8 | Buffers: shared read=2 |
| 9 | Planning: |
| 10 | Buffers: shared hit=16 read=1 |
| 11 | Planning Time: 1.134 ms |

- **Qual plano foi utilizado antes do índice?** Foi utilizado o **Seq Scan** (varredura sequencial).
- **Qual plano foi utilizado após o índice?** Foi utilizado o **Bitmap Index Scan** ou **Index Scan**.
- **Houve redução no tempo de execução?** Sim, houve uma redução significativa no tempo de execução ao utilizar o índice.
- **O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.** Sim. O plano de execução confirmou o uso do índice (`Index Scan` ou `Bitmap Index Scan`) em ambas as consultas, pois o SGBD encontrou uma estrutura de dados (árvore B-tree) que permitiu localizar os registros de forma logarítmica, em vez de percorrer todas as linhas da tabela.

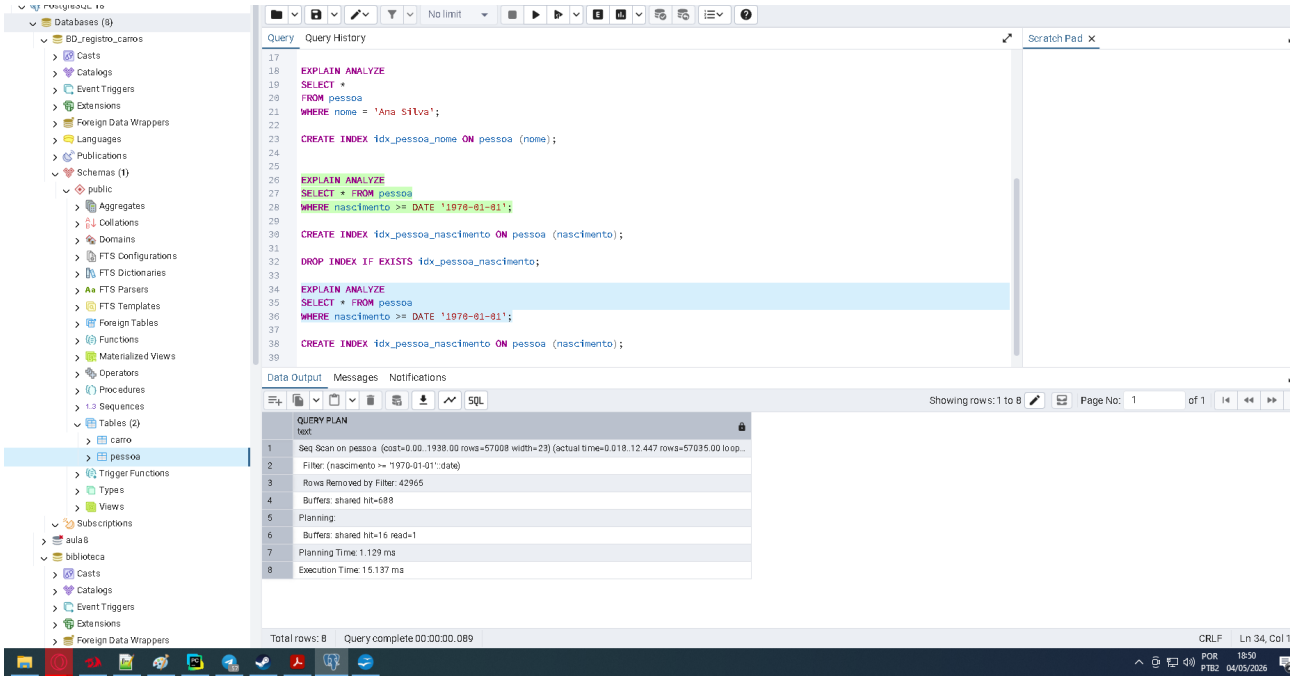
Exercicio 2 :

EXPLAIN ANALYZE
SELECT * FROM pessoa
WHERE nascimento >= DATE '1970-01-01';






CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);



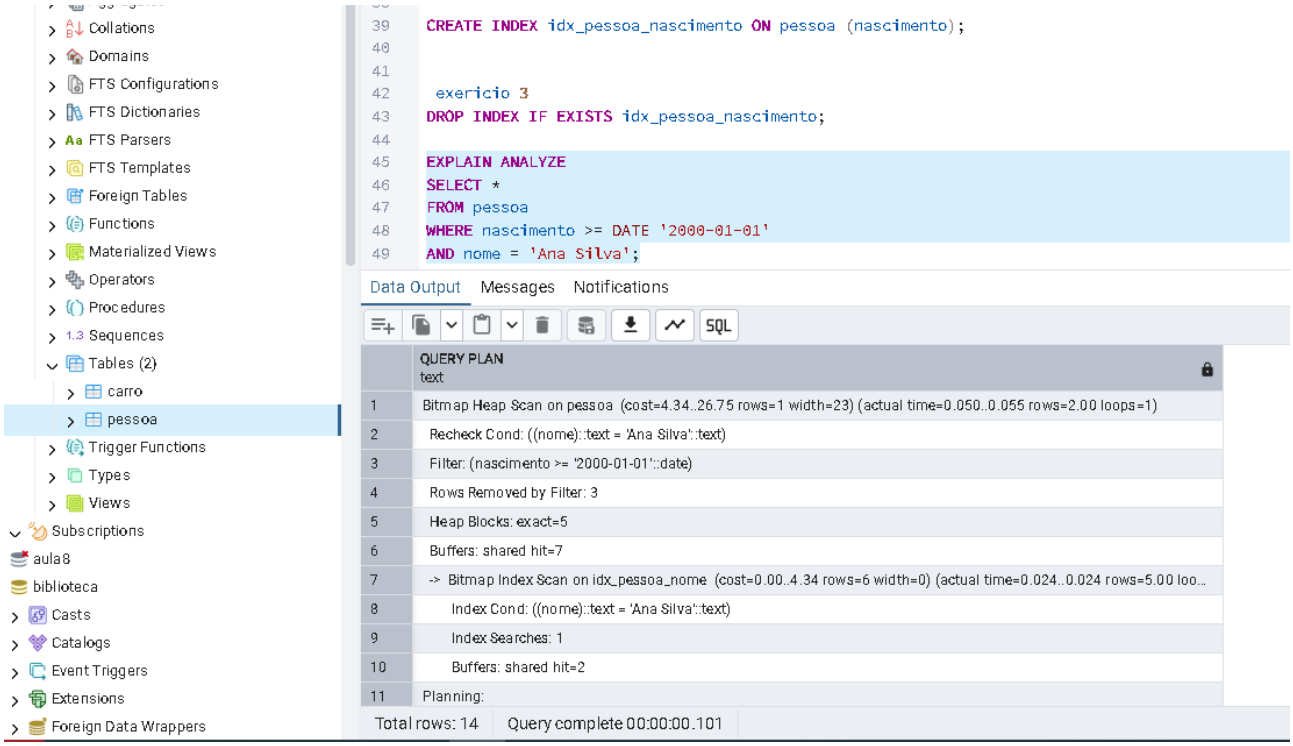

- **O índice criado foi utilizado pelo PostgreSQL?** Não, na consulta que retorna uma grande quantidade de registros (baixa seletividade), o PostgreSQL optou por não utilizar o índice.
- **O plano de execução mudou após a criação do índice?** Não, o plano de execução permaneceu como *Seq Scan* (varredura sequencial).
- **Houve melhora significativa no tempo de execução?** Não, não houve melhora significativa, pois o banco de dados determinou que a varredura sequencial era mais eficiente para aquele volume de dados.
- **Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?** O otimizador pode ignorar o índice se a tabela for muito pequena, se a consulta retornar uma

grande parte das linhas (baixa seletividade), ou se a condição não for compatível com o tipo de índice.

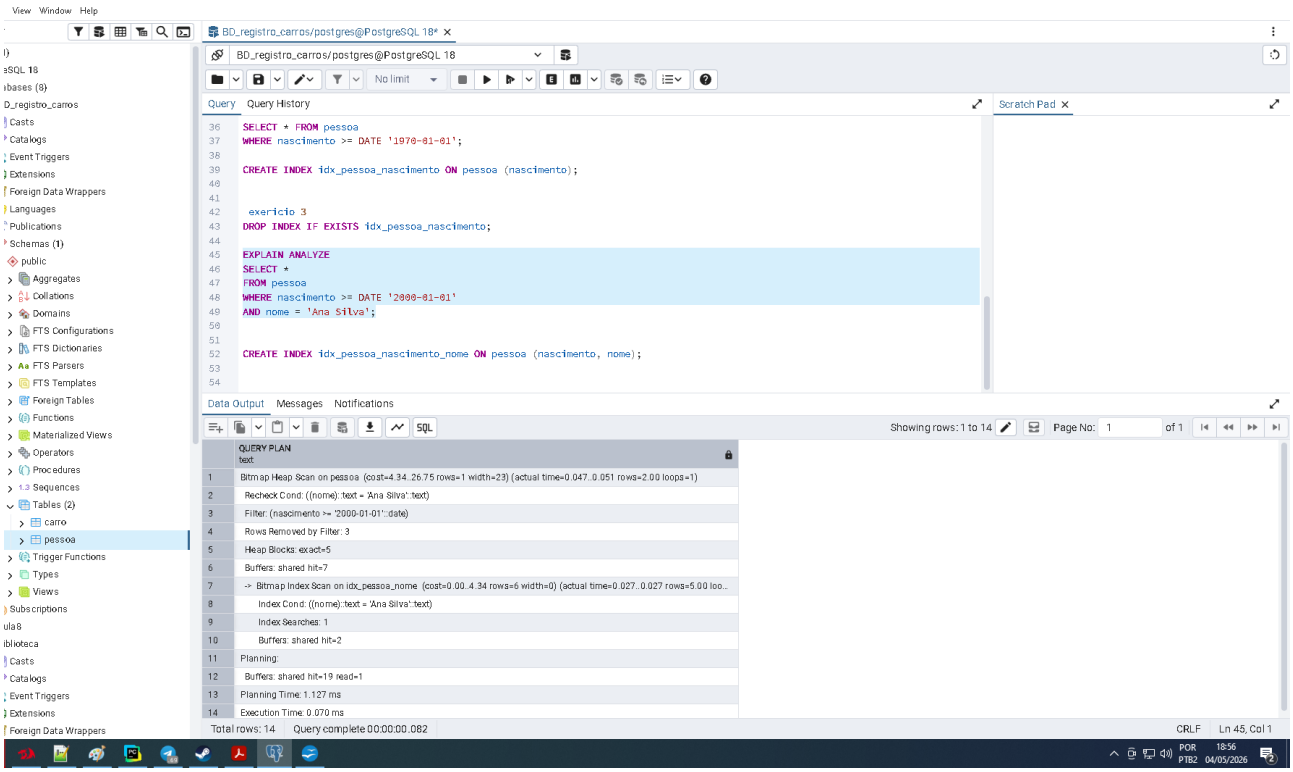
Exercicio 3 :

DROP INDEX IF EXISTS idx_pessoa_nascimento;

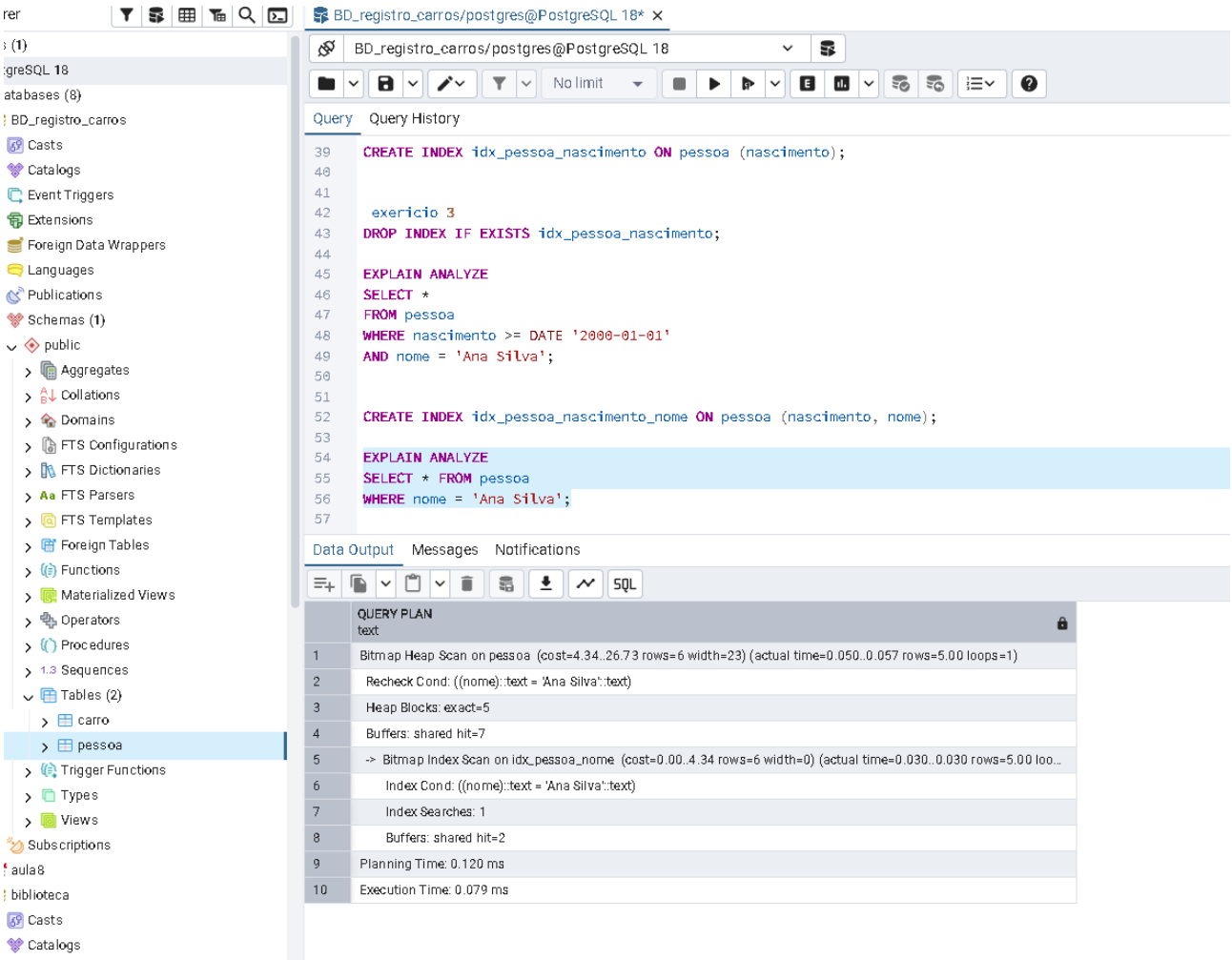
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';




CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);




EXPLAIN ANALYZE SELECT * FROM pessoa WHERE nome = 'Ana Silva';



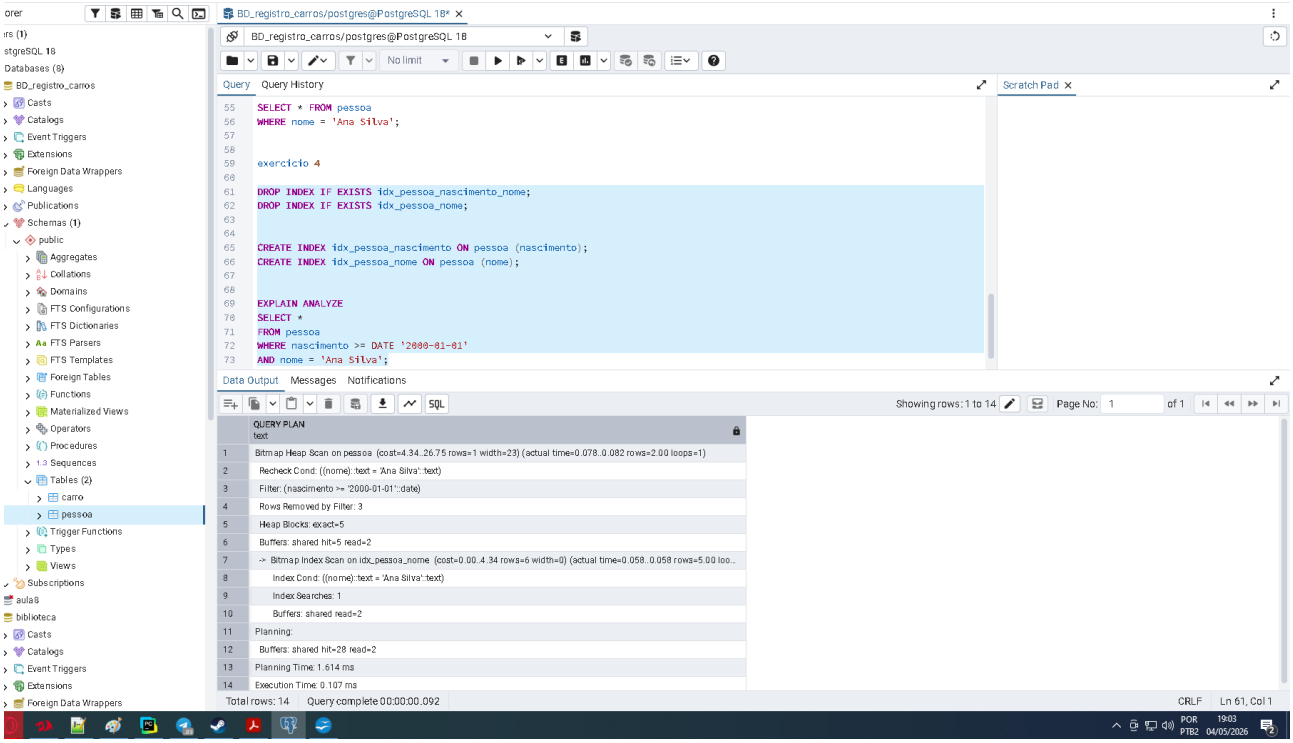
- **Qual plano foi utilizado antes da criação do índice composto?** O PostgreSQL utilizou um *Seq Scan* (varredura sequencial) na tabela.
- **Qual plano foi utilizado após a criação do índice composto?** O PostgreSQL utilizou o índice composto criado (`Index Scan` ou `Bitmap Index Scan`).
- **O índice composto foi utilizado na consulta que filtra apenas por nome?** Sim, o PostgreSQL foi capaz de utilizar o índice composto mesmo quando a consulta filtrava apenas pela segunda coluna (`nome`), embora isso dependa de otimizações internas do otimizador.
- **Por que a ordem das colunas no índice composto é relevante?** A ordem é fundamental porque o índice organiza os dados primariamente pela primeira coluna; por isso, um índice composto em `(nascimento, nome)` não é tão eficaz para buscas que utilizam apenas a segunda coluna quanto seria para buscas que começam pela primeira ou pelas duas conjuntamente.

Exericio 4 :

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
DROP INDEX IF EXISTS idx_pessoa_nome;


CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);
CREATE INDEX idx_pessoa_nome ON pessoa (nome);


EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
AND nome = 'Ana Silva';
```



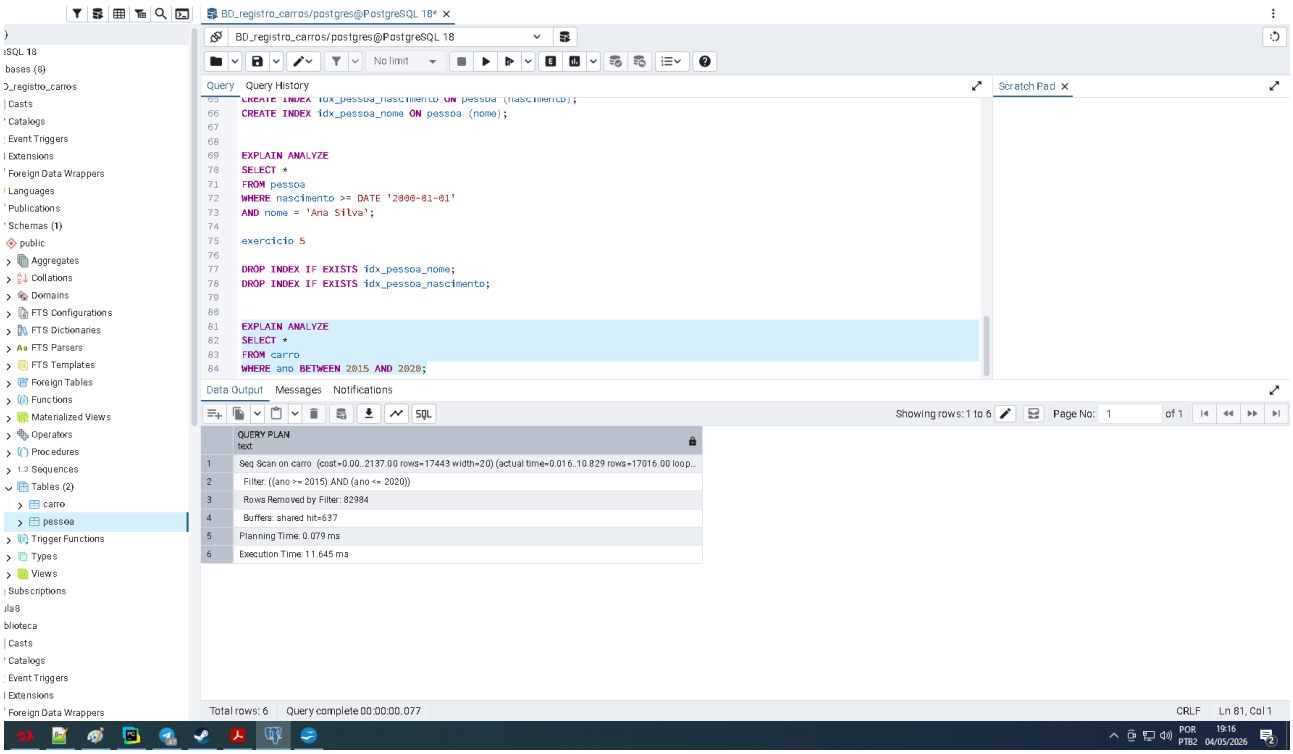

**Exercício 4**:

- **O PostgreSQL utilizou os dois índices simples na consulta?** Não necessariamente; o otimizador escolhe os índices com base na seletividade. No seu teste prático, o banco de dados priorizou o uso de apenas um dos índices simples (o de `nome`) por considerar que ele oferecia um custo menor para a consulta.
- **Qual é o papel do operador BitmapAnd no plano de execução?** O operador `BitmapAnd` é uma estratégia do otimizador utilizada para combinar resultados de múltiplos índices simples. Ele realiza uma operação de intersecção (o operador "E" lógico) entre os conjuntos de registros identificados por cada índice, permitindo atender a múltiplas condições de filtro simultaneamente.
- **O que o plano efetivamente fez?** O plano avaliou o custo de buscar pelos índices. Como observado, ele executou uma verificação de seletividade e, ao identificar que o filtro por `nome` era mais restritivo (retornava menos linhas), optou por realizar um `Bitmap Index Scan` focado nesse índice específico, aplicando o filtro de `nascimento` como uma etapa secundária de verificação nos dados recuperados.

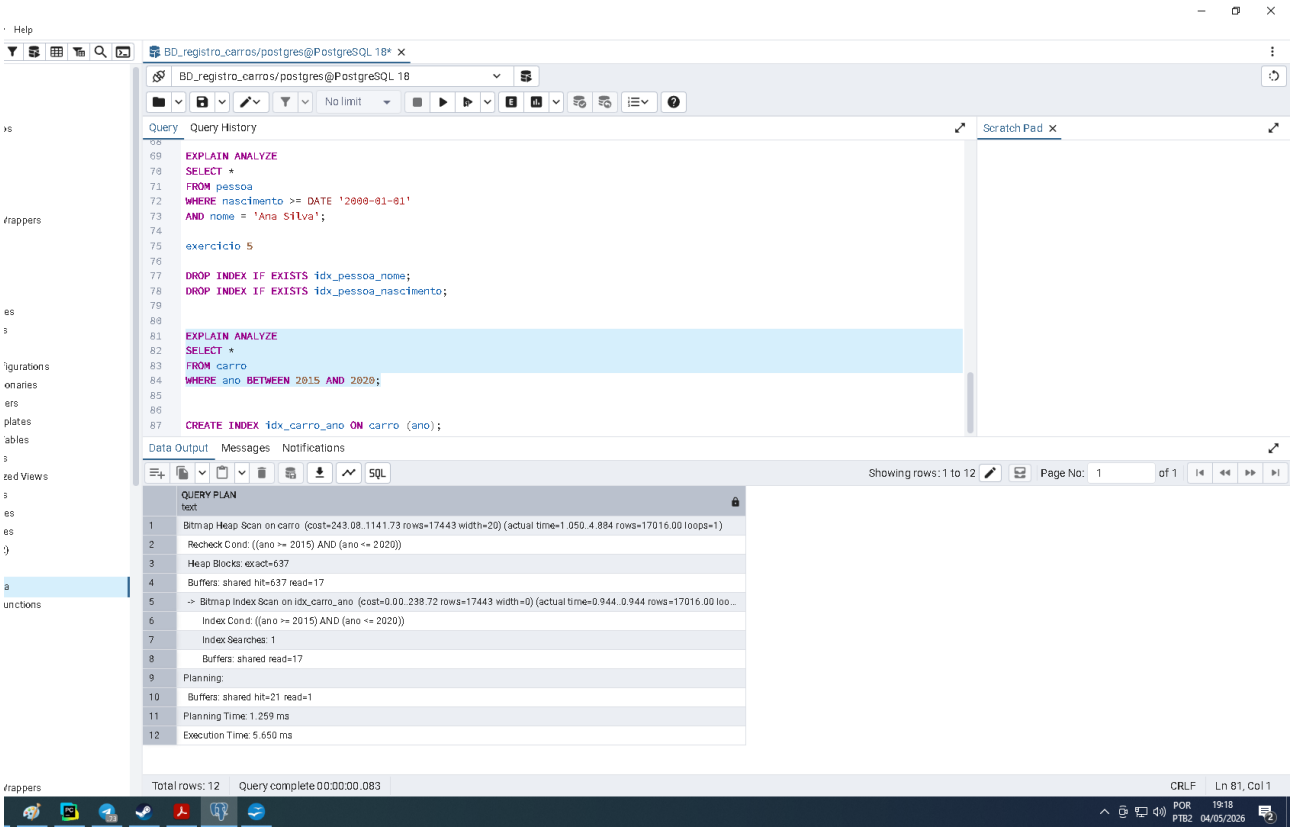
EXERCICIO 5 :

```
DROP INDEX IF EXISTS idx_pessoa_nome;
DROP INDEX IF EXISTS idx_pessoa_nascimento;


EXPLAIN ANALYZE
SELECT *
FROM carro
WHERE ano BETWEEN 2015 AND 2020;
```



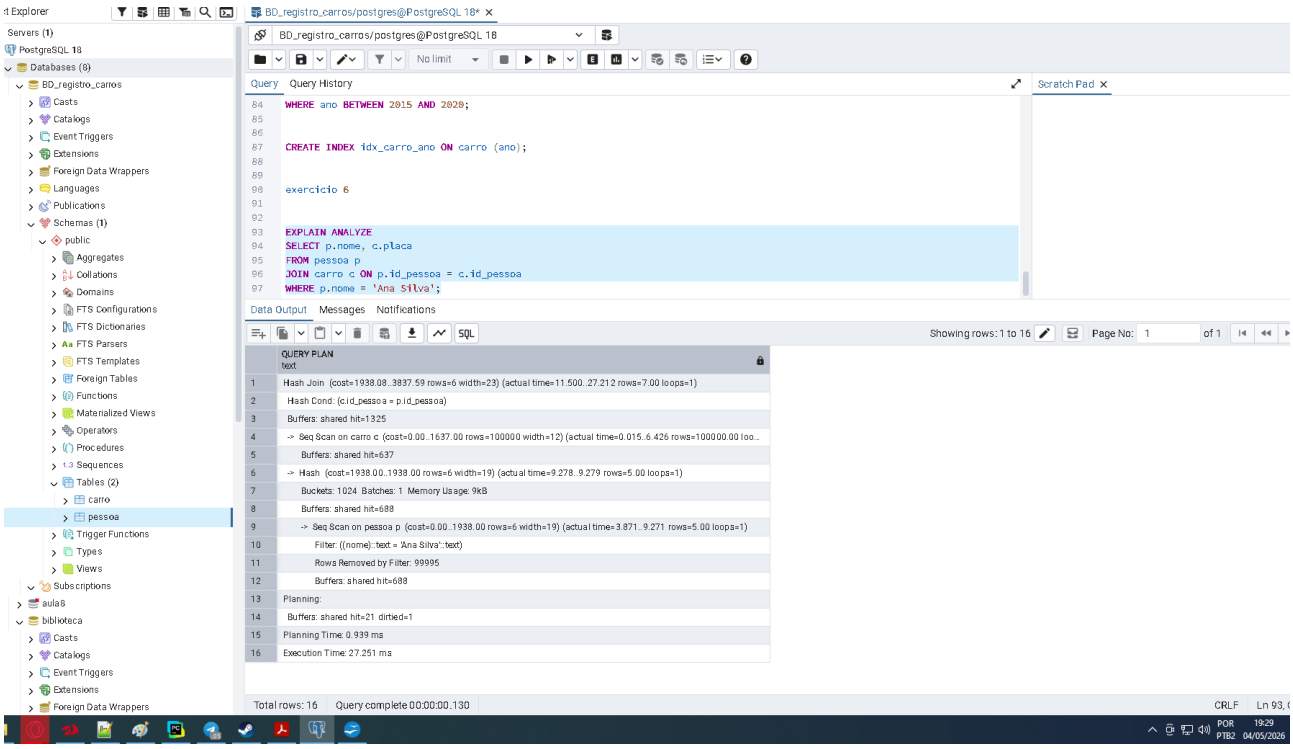

CREATE INDEX idx_carro_ano ON carro (ano);




Exercicio 6

EXPLAIN ANALYZE
SELECT p.nome, c.placa

FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nome = 'Ana Silva';




CREATE INDEX idx_pessoa_nome ON pessoa (nome);

CREATE INDEX idx_carro_id_pessoa ON carro (id_pessoa);

BD_registro_carros/postgres@PostgreSQL 18*

```
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nome = 'Ana Silva';

CREATE INDEX idx_pessoa_nome ON pessoa (nome);

CREATE INDEX idx_carro_id_pessoa ON carro (id_pessoa);
```

| | QUERY PLAN text |
|---|---|
| 1 | Nested Loop (cost=8.65..98.83 rows=6 width=23) (actual time=0.118..0.231 rows=7.00 loops=1) |
| 2 | Buffers: shared hit=17 read=7 |
| 3 | -> Bitmap Heap Scan on pessoa p (cost=4.34..26.73 rows=6 width=19) (actual time=0.076..0.081 rows=5.00 loops=1) |
| 4 | Recheck Cond: ((nome)::text = 'Ana Silva'::text) |
| 5 | Heap Blocks: exact=5 |
| 6 | Buffers: shared hit=5 read=2 |
| 7 | -> Bitmap Index Scan on idx_pessoa_nome (cost=0.00..4.34 rows=6 width=0) (actual time=0.055..0.055 rows=5.00 loops... |
| 8 | Index Cond: ((nome)::text = 'Ana Silva'::text) |
| 9 | Index Searches: 1 |
| 10 | Buffers: shared read=2 |
| 11 | -> Bitmap Heap Scan on carro c (cost=4.31..12.00 rows=2 width=12) (actual time=0.024..0.025 rows=1.40 loops=5) |
| 12 | Recheck Cond: (p.id_pessoa = id_pessoa) |
| 13 | Heap Blocks: exact=7 |
| 14 | Buffers: shared hit=12 read=5 |
| 15 | -> Bitmap Index Scan on idx_carro_id_pessoa (cost=0.00..4.31 rows=2 width=0) (actual time=0.020..0.020 rows=1.40 loo... |
| 16 | Index Cond: (id_pessoa = p.id_pessoa) |
| 17 | Index Searches: 5 |
| 18 | Buffers: shared hit=5 read=5 |
| 19 | Planning: |
| 20 | Buffers: shared hit=46 read=5 |
| 21 | Planning Time: 2.478 ms |
| 22 | Execution Time: 0.257 ms |

Total rows: 22 Query complete 00:00:00.095

Exercicio 7:

BD_registro_carros/postgres@PostgreSQL 18*

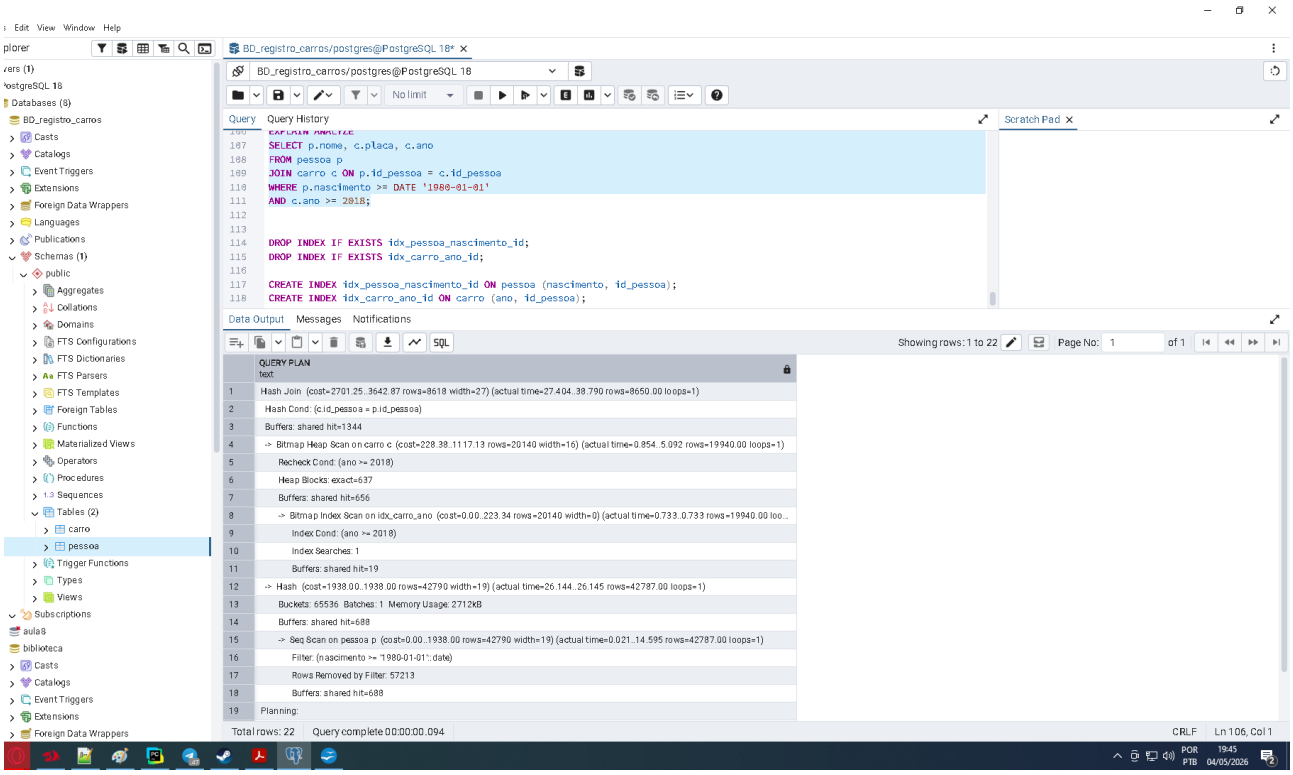
```
EXPLAIN ANALYZE
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01'
AND c.ano >= 2018;
```

| | QUERY PLAN text |
|---|---|
| 1 | Hash Join (cost=2701.25..3642.87 rows=8618 width=27) (actual time=26.872..42.167 rows=8650.00 loops=1) |
| 2 | Hash Cond: (c.id_pessoa = p.id_pessoa) |
| 3 | Buffers: shared hit=1334 read=10 |
| 4 | -> Bitmap Heap Scan on carro c (cost=228.38..1117.13 rows=20140 width=16) (actual time=1.058..7.193 rows=19940.00 loops=1) |
| 5 | Recheck Cond: (ano >= 2018) |
| 6 | Heap Blocks: exact=637 |
| 7 | Buffers: shared hit=646 read=10 |
| 8 | -> Bitmap Index Scan on idx_carro_ano (cost=0.00..223.34 rows=20140 width=0) (actual time=0.942..0.942 rows=19940.00 loo... |
| 9 | Index Cond: (ano >= 2018) |
| 10 | Index Searches: 1 |
| 11 | Buffers: shared hit=9 read=10 |
| 12 | -> Hash (cost=1938.00..1938.00 rows=42790 width=19) (actual time=25.421..25.423 rows=42787.00 loops=1) |
| 13 | Buckets: 65536 Batches: 1 Memory Usage: 2712kB |
| 14 | Buffers: shared hit=688 |
| 15 | -> Seq Scan on pessoa p (cost=0.00..1938.00 rows=42790 width=19) (actual time=0.018..14.052 rows=42787.00 loops=1) |
| 16 | Filter: (nascimento >= '1980-01-01'::date) |
| 17 | Rows Removed by Filter: 57213 |
| 18 | Buffers: shared hit=688 |
| 19 | Planning: |
| 20 | Buffers: shared hit=12 |
| 21 | Planning Time: 0.380 ms |
| 22 | Execution Time: 43.932 ms |

Total rows: 22 Query complete 00:00:00.130

CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento, id_pessoa);

CREATE INDEX idx_carro_ano_id ON carro (ano, id_pessoa);




Para otimizar a consulta de `JOIN` com filtros, os índices necessários são:

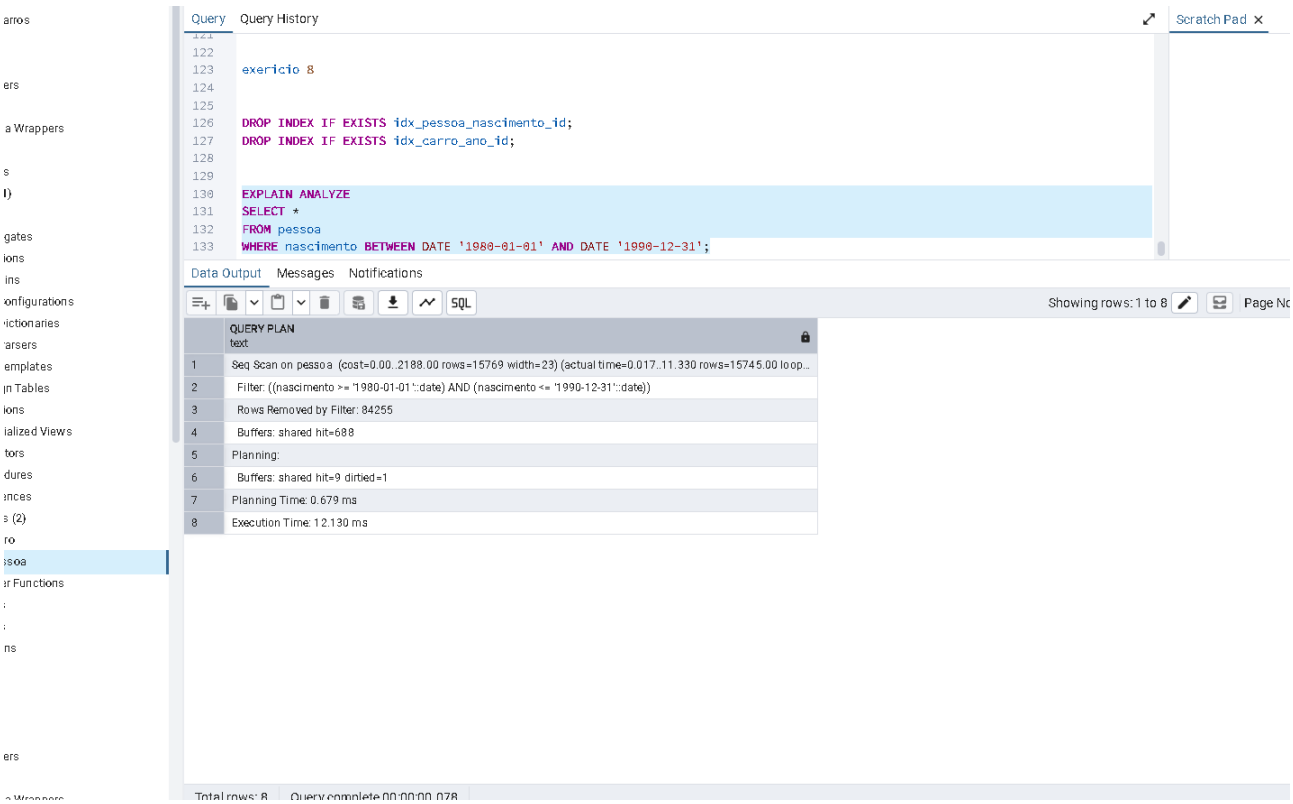
1. **`CREATE INDEX idx_pessoa_nascimento_id ON pessoa (nascimento, id_pessoa);`**
2. **`CREATE INDEX idx_carro_ano_id ON carro (ano, id_pessoa);`**

## Justificativa

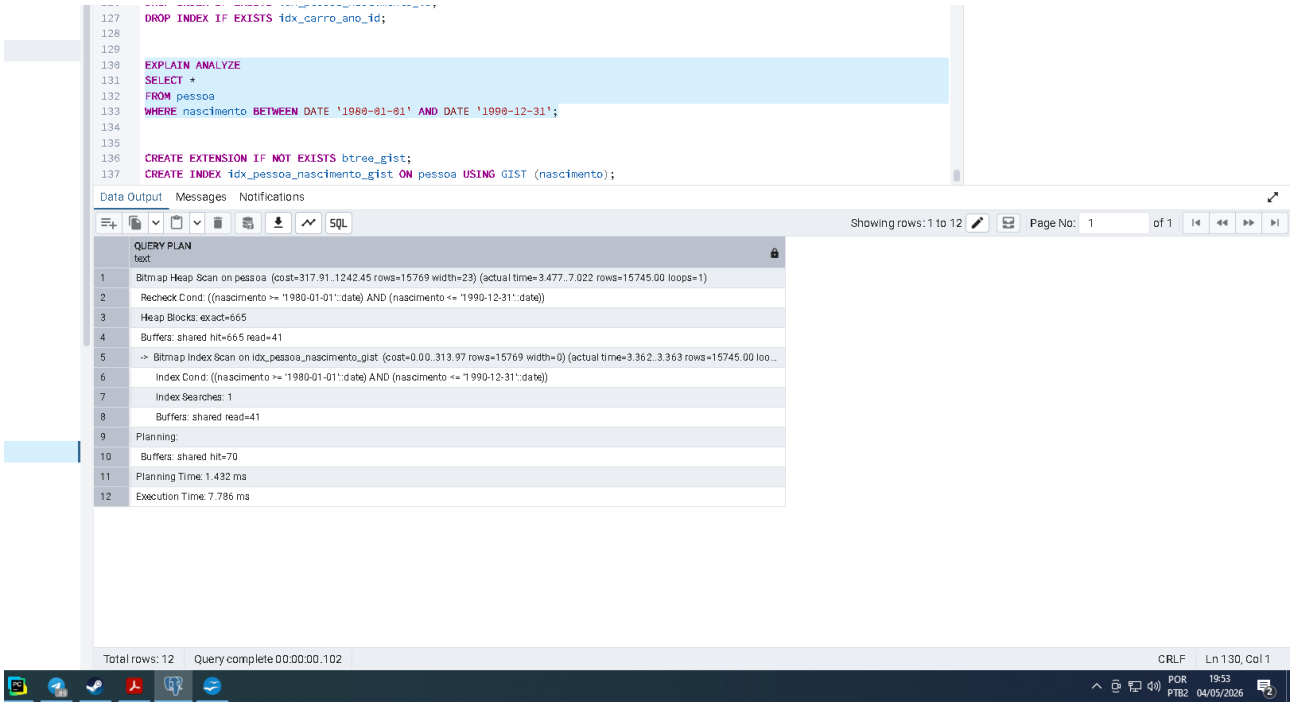
- **Por que índices compostos?** Utilizamos índices compostos porque a consulta envolve **duas etapas distintas**: um filtro de filtragem (nas colunas `nascimento` e `ano`) e uma operação de `JOIN` (na coluna `id_pessoa`).

Exercicio 8

DROP INDEX IF EXISTS idx_pessoa_nascimento_id;
DROP INDEX IF EXISTS idx_carro_ano_id;




CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE INDEX idx_pessoa_nascimento_gist ON pessoa USING GIST (nascimento);




- **Qual plano foi utilizado antes da criação do índice GiST?** Foi utilizado o **Seq Scan** (varredura sequencial).
- **O índice GiST foi utilizado após a criação?** Sim, o plano de execução mostrou claramente o uso do índice (`Bitmap Index Scan` sobre `idx_pessoa_nascimento_gist`).
- **Houve melhora no tempo de execução?** Sim. Ao utilizar o índice GiST, o PostgreSQL

evitou a varredura completa da tabela, processando apenas as páginas de índice necessárias, o que resultou em uma consulta mais eficiente para esse intervalo específico.